

OLD DOMINION
UNIVERSITY

CCL Group Training

OpenFOAM Tutorial: runnin an aachemBomb simulation on HPC

Date 10/02/2026

ODU Wahab's Container

ODU Wahab's Container Runner Wrapper (commonly invoked using the crun command) is the execution interface used on Old Dominion University's Wahab HPC cluster to launch scientific software and utilities.

Rather than compiling complex tools directly on the shared cluster operating system, ODU's HPC admins package utilities—such as Python distributions, PyTorch, AlphaFold, R, or bioinformatic tools—into lightweight container images (often built with Singularity/Apptainer). The crun utility acts as the middleman between your command line, the host file system, and the isolated software container.

How it Works Behind the Scenes:

1. **Environment Setup:** When you run module load container_env <utility> or module load <utility>, Environment Modules set up pathing and environment variables pointing to a wrapper script (crun).

2. **Container Launch:** When you run a command using crun, it starts the containerized tool while automatically mounting (binding) host system directories like your ~/home directory, scratch spaces, and cluster hardware devices (such as NVIDIA V100 GPUs or Intel MKL math libraries).

3. **Transparent Execution:** It feeds your input arguments directly into the container and sends output directly back to your terminal screen or log files.

ODU Wahab's Container

Common Workflow Examples:

1. **Basic Utility Execution**
   To run a script using a containerized module:

```
module load container_env python3
crun python3 my_script.py
```

2. **Application-Specific Wrappers**
   Some loaded modules automatically configure specific alias wrappers:

```
module load pytorch
crun.pytorch-gpu my_model.py
```

3. **Interactive Shells & Container Package Management**
   If you need to view installed dependencies or run interactive commands inside the container space:

```
# Check installed packages inside the container environment
crun pip list

# Specify user environment overlay paths (if using custom virtual environments)
crun -p ~/envs/my_proj pip list
```

OpenFOAM in Container Environment

OpenFOAM on Wahab is provided as a containerized module:
OpenFOAM has complex system dependencies (specific C++ compilers, MPI versions, and custom environment settings). Packaging it into a container prevents conflicts with the underlying host OS and allows you to run modern CFD solvers without altering the cluster environment.

How OpenFOAM Works with crun?
When you load the OpenFOAM environment, its solvers and native utilities (like spraFoam, or blockMesh) need to be invoked through the container runner (crun).

```
module load container_env openfoam
crun blockMesh
crun simpleFoam
```

Running Parallel Jobs via Slurm?

```
module load container_env openfoam

# Decompose domain
crun decomposePar

# Run parallel solver using MPI with crun
srun crun simpleFoam -parallel
```

ALERT:
Every execution requires preprending crun to every single command.

OpenFOAM in Container Environment

crun bash command:
This command launches an **interactive bash shell directly inside the OpenFOAM container environment**, with all of OpenFOAM's specific environment variables ($WM_PROJECT_DIR, $FOAM_USER_APPBIN, solver paths, and aliases) pre-loaded and active.

By entering an interactive container shell via **crun bash**, you enter a sub-shell operating *inside* the container context:

1. **No More crun Prefixing:** You can run native OpenFOAM commands, aliases, and scripts directly (blockMesh, sprayFoam, paraFoam) just as if you were working on a standard desktop installation.

2. **Pre-Sourced Environment:** The openfoam.com/2512 module automatically sources OpenFOAM's inner etc/bashrc upon entering the crun bash shell, populating all required environment variables automatically.

3. **Interactive Debugging & Scripting:** It lets you inspect $FOAM_APP, test custom C++ solvers with wmake, or step through tutorials iteratively without the overhead of starting and stopping a new container instance for every minor utility call.

OpenFOAM in Container Environment

Key Workflow Overview

This command launches an **interactive bash shell directly inside the OpenFOAM container environment**, with all of OpenFOAM's specific environment variables ($WM_PROJECT_DIR, $FOAM_USER_APPBIN, solver paths, and aliases) pre-loaded and active.

```
# 1. Load the OpenFOAM environment module
module load openfoam.com/2512

# 2. Enter the interactive container shell
crun bash

# 3. Work directly inside the container (Notice your terminal prompt may change)
blockMesh
simpleFoam

# 4. Exit the container shell when finished
exit
```

Note on Slurm Jobs: While crun bash is great for interactive terminal sessions on head/interactive nodes, in batch script submissions (.slurm files), prepending individual steps like srun crun simpleFoam -parallel is still preferred to allow Slurm to track process IDs and resource usage directly.

Launching sprayFoam on Wahab

ALERT: Wahab Containerized OpenFOAM omits tutorials directory

Download the source archive directly via official release tarball

```
[s3kim@wahab-01]$ wget https://dl.openfoam.com/source/v2512/OpenFOAM-v2512.tgz
[s3kim@wahab-01]$ tar xvfz OpenFOAM-v2512.tgz
```

Clone from official GitHub Mirror

```
[s3kim@wahab-01]$ git clone --depth 1 --branch OpenFOAM-v2512
https://github.com/OpenFOAM/OpenFOAM-v2512.git ./
```

Launching sprayFoam on Wahab

Launching an Interactive Shell inside the Container

To open an interactive shell inside the OpenFOAM container so you can inspect $FOAM_TUTORIALS and copy the tutorial case.

```
[s3kim@wahab-01 ~]$ module load openfoam.com/2512
[s3kim@wahab-01 ~]$ crun bash
ERROR: Unable to locate a modulefile for 'container_env'
ERROR: Unable to locate a modulefile for 'python3'
Apptainer>
```

Why the errors appear:

When you run crun bash, crun initializes the shell environment inside the container by reading initialization files (such as .bashrc or module init scripts). These scripts attempt to run module load container_env python3 by default as part of the startup configuration on Wahab.

Why it fails harmlessly:

Inside the Apptainer container, the underlying host environment module trees (container_env and python3) do not exist—nor do they need to, because OpenFOAM comes pre-packaged with its own required binaries, libraries, and C++ compilers inside the image.

(tested on 10/02/2026)

Launching sprayFoam on Wahab

Launching an Interactive Shell inside the Container

Once inside the container shell, all OpenFOAM paths are set up. Copy the tutorial case directly into your scratch directory.

```
Apptainer> mkdir -p /scratch/s3kim/OpenFOAM/run
Apptainer> cd /scratch/s3kim/OpenFOAM/run
```

```
Apptainer> cp -r $FOAM_TUTORIALS/combustion/sprayFoam/aachenBomb .
cp: cannot stat '/usr/lib/openfoam/openfoam2512/tutorials/combustion/sprayFoam/aachenBomb': No such file
```

For this tutorial, I've prepared the tutorial director from the official openfoam.com repository in the following location:

```
Apptainer> cp -r /scratch/s3kim/OpenFOAM/OpenFOAM-v2512/tutorials/lagrangian/sprayFoam/aachenBomb/ ./
Apptainer> cd aachenBomb/
```

Now your aachemBomb case is prepared for your simulation. Enjoy your CFD!